

C++ Programming: From Problem Analysis to Program Design, Sixth Edition

Chapter 9: Records (`structs`)

Objectives

In this chapter, you will:

- Learn about records (`structs`)
- Examine various operations on a `struct`
- Explore ways to manipulate data using a `struct`
- Learn about the relationship between a `struct` and functions
- Discover how arrays are used in a `struct`
- Learn how to create an array of `struct` items

Records (`structs`)

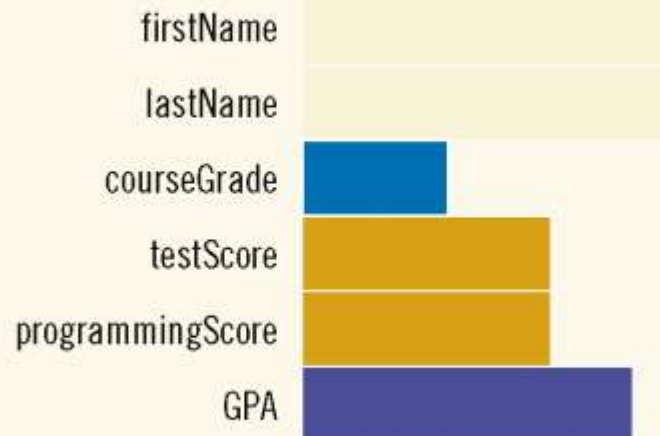
- struct: collection of a fixed number of components (members), accessed by name
- A `struct` is a definition, not a declaration
 - Members may be of different types
- Syntax:

```
struct structName
{
    dataType1 identifier1;
    dataType2 identifier2;
    .
    .
    .
    dataTypeN identifierN;
};
```

```
struct studentType
{
    string firstName;
    string lastName;
    char courseGrade;
    int testScore;
    int programmingScore;
    double GPA;
};

//variable declaration
studentType newStudent;
studentType student;
```

newStudent



student

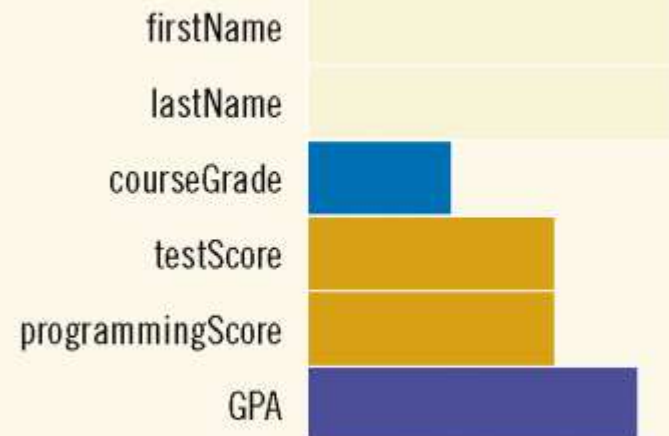


FIGURE 11-1 `struct newStudent` and `student`

Accessing `struct` Members

- The syntax for accessing a `struct` member is:

```
structVariableName.memberName
```

- The dot (.) is an operator, called the member access operator

Accessing `struct` Members (continued)

- To initialize the members of `newStudent`:

```
newStudent.GPA = 0.0;  
newStudent.firstName = "John";  
newStudent.lastName = "Brown";
```

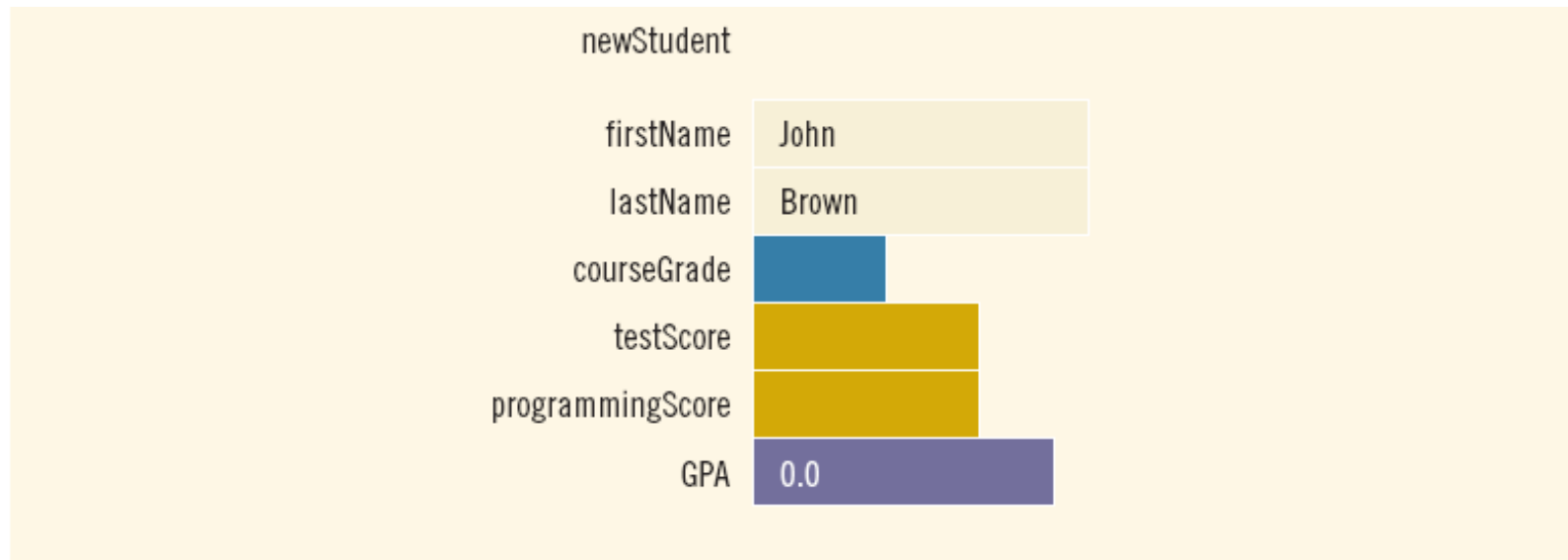


FIGURE 11-2 `struct newStudent`

Accessing `struct` Members (continued)

- More examples:

```
cin >> newStudent.firstName;
cin>>newStudent.testScore>>newStudent.programmingScore;
score = (newStudent.testScore +
        newStudent.programmingScore) / 2;
if (score >= 90)
    newStudent.courseGrade = 'A';
else if (score >= 80)
    newStudent.courseGrade = 'B';
else if (score >= 70)
    newStudent.courseGrade = 'C';
else if (score >= 60)
    newStudent.courseGrade = 'D';
else
    newStudent.courseGrade = 'F';
```

Assignment

- Value of one `struct` variable can be assigned to another `struct` variable of the same type using an assignment statement
- The statement:

```
student = newStudent;
```

copies the contents of `newStudent` **into**
`student`

Assignment (continued)

- The assignment statement:

```
student = newStudent;
```

is equivalent to the following statements:

```
student.firstName = newStudent.firstName;  
student.lastName = newStudent.lastName;  
student.courseGrade = newStudent.courseGrade;  
student.testScore = newStudent.testScore;  
student.programmingScore =  
    newStudent.programmingScore;  
student.GPA = newStudent.GPA;
```

Comparison (Relational Operators)

- Compare `struct` variables member-wise
 - No aggregate relational operations allowed
- To compare the values of `student` and `newStudent`:

```
if (student.firstName == newStudent.firstName &&  
    student.lastName == newStudent.lastName)
```

```
.*  
.*  
.*
```

Input/Output

- No aggregate input/output operations on a `struct` variable
- Data in a `struct` variable must be read one member at a time
- The contents of a `struct` variable must be written one member at a time

```
cout << newStudent.firstName << " " << newStudent.lastName  
    << " " << newStudent.courseGrade  
    << " " << newStudent.testScore  
    << " " << newStudent.programmingScore  
    << " " << newStudent.GPA << endl;
```

struct Variables and Functions

- A `struct` variable can be passed as a parameter by value or by reference

```
void printStudent(studentType student)
{
    cout << student.firstName << " " << student.lastName
        << " " << student.courseGrade
        << " " << student.testScore
        << " " << student.programmingScore
        << " " << student.GPA << endl;
}
```

- A function can return a value of type `struct`

Arrays versus `structs`

TABLE 11-1 Arrays vs. `structs`

Aggregate Operation	Array	<code>struct</code>
Arithmetic	No	No
Assignment	No	Yes
Input/output	No (except strings)	No
Comparison	No	No
Parameter passing	By reference only	By value or by reference
Function returning a value	No	Yes

Arrays in `structs`

- Two key items are associated with a list:
 - Values (elements)
 - Length of the list
- Define a `struct` containing both items:

```
const int ARRAY_SIZE = 1000;

struct listType
{
    int listElem[ARRAY_SIZE];    //array containing the list
    int listLength;              //length of the list
};
```

Arrays in structs [Cont.]

```
listType intList;
```

```
const int ARRAY_SIZE = 1000;  
  
struct listType  
{  
    int listElem[ARRAY_SIZE];    //array containing the list  
    int listLength;              //length of the list  
};
```

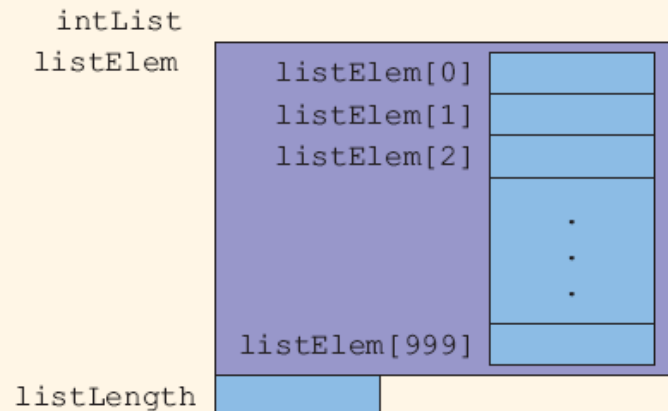


FIGURE 11-5 struct variable intList

Arrays in structs (cont'd.)

```
intList.listLength = 0;           //Line 1
intList.listElem[0] = 12;         //Line 2
intList.listLength++;             //Line 3
intList.listElem[1] = 37;         //Line 4
intList.listLength++;             //Line 5
```

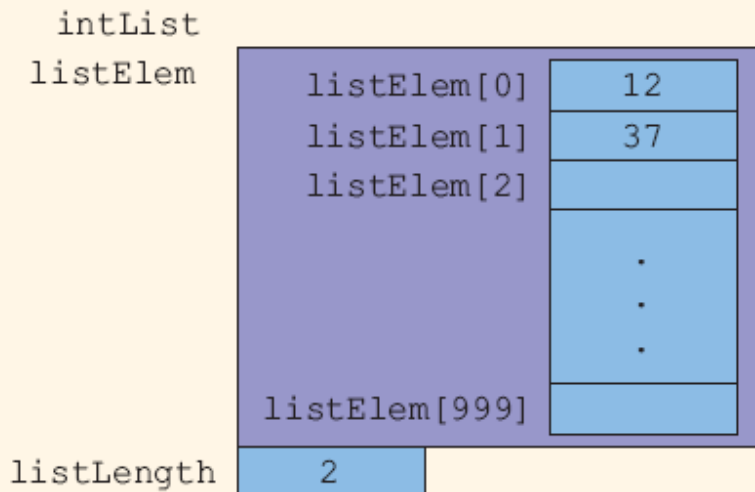


FIGURE 11-6 `intList` after the statements in Lines 1 through 5 execute

Arrays of structs

```
struct employeeType
{
    string firstName;
    string lastName;
    int    personID;
    string deptID;
    double yearlySalary;
    double monthlySalary;
    double yearToDatePaid;
    double monthlyBonus;
};
```

```
employeeType employees[50];
```

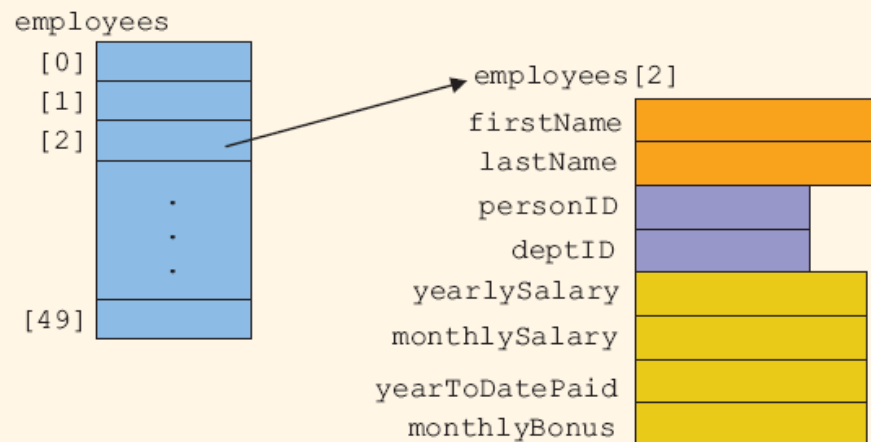


FIGURE 11-7 Array of employees

Arrays of structs [Cont.]

```
int main()
{
    employeeType employees[50];

    for (int i=0; i<50; i++)
    {
        employees[i].firstName= "EMP";
    }

    for (int i=0; i<50; i++)
    {
        cout<< employees[i].firstName<< " ";
    }

    return 0;
}
```

```
#include<iostream>
#include<string>
using namespace std;

struct employeeType
{
    string firstName;
    string lastName;
    int personID;
    string deptID;
    double yearlySalary;
    double monthlySalary;
    double yearToDatePaid;
    double monthlyBonus;
};
```

structs within a struct

```
struct employeeType
{
    string firstname;
    string middlename;
    string lastname;
    string empID;
    string address1;
    string address2;
    string city;
    string state;
    string zip;
    int hiremonth;
    int hireday;
    int hireyear;
    int quitmonth;
    int quitday;
    int quityear;
    string phone;
    string cellphone;
    string fax;
    string pager;
    string email;
    string deptID;
    double salary;
};
```

versus

```
struct addressType
{
    string address1;
    string address2;
    string city;
    string state;
    string zip;
};

struct dateType
{
    int month;
    int day;
    int year;
};

struct contactType
{
    string phone;
    string cellphone;
    string fax;
    string pager;
    string email;
};

struct nameType
{
    string first;
    string middle;
    string last;
};

struct employeeType
{
    nameType name;
    string empID;
    addressType address;
    dateType hireDate;
    dateType quitDate;
    contactType contact;
    string deptID;
    double salary;
};
```

```
struct nameType
{
    string first;
    string middle;
    string last;
};
```

```
struct addressType
{
    string address1;
    string address2;
    string city;
    string state;
    string zip;
};
```

```
struct dateType
{
    int month;
    int day;
    int year;
};
```

```
struct contactType
{
    string phone;
    string cellphone;
    string fax;
    string pager;
    string email;
};
```

```
struct employeeType
```

```
{
    nameType name;
    string empID;
    addressType address;
    dateType hireDate;
    dateType quitDate;
    contactType contact;
    string deptID;
    double salary;
};
```

```
int main()
{
    employeeType newEmployee;

    newEmployee.hireDate.day=5;
    newEmployee.hireDate.month=1;
    newEmployee.hireDate.year=2017;
    cout<<newEmployee.hireDate.day<<"-"<<newEmployee.hireDate.month
    <<newEmployee.hireDate.year<<endl;
}
```

```
employeeType newEmployee;
```

newEmployee

name	first	
	middle	
	last	
empID		
address	address 1	
	address 2	
	city	
	state	
	zip	
hireDate	month	
	day	
	year	
quitDate	month	
	day	
	year	
contact	phone	
	cellphone	
	fax	
	pager	
	email	
deptID		
salary		

FIGURE 11-8 `struct` variable newEmployee



The End